```
    Iterator block_start=first;
    for(unsigned long i=0;i<(num_threads-1);++i)
    {
        Iterator block_end=block_start;
        std::advance(block_end,block_size);
        std::packaged_task<void(void)> task(      <-- 2
            [=]()
            {
                std::for_each(block_start,block_end,f);
            });
        futures[i]=task.get_future();
        threads[i]=std::thread(std::move(task));      <-- 3
        block_start=block_end;
    }
    std::for_each(block_start,last,f);
    for(unsigned long i=0;i<(num_threads-1);++i)
    {
        futures[i].get();      <-- 4
    }
}
```

The basic structure of the code is identical to that of listing 8.4, which is unsurprising. The key difference is that the `futures` vector stores `std::future<void>` ❶ because the worker threads don't return a value, and a simple lambda function that invokes the function `f` on the range from `block_start` to `block_end` is used for the task ❷. This avoids having to pass the range into the thread constructor ❸. Since the worker threads don't return a value, the calls to `futures[i].get()` ❹ just provide a means of retrieving any exceptions thrown on the worker threads; if you don't wish to pass on the exceptions, you could omit this.

Just as your parallel implementation of `std::accumulate` could be simplified using `std::async`, so can your `parallel_for_each`. Such an implementation follows.

Listing 8.8. A parallel version of `std::for_each` using `std::async`